Lead-value model for Fonio's ad bidding

Case study write-up, Marketing Engineer. The notebook, code and all numbers are in the repository (README explains what is where).

Today Meta and Google Ads optimise for signups, and a Meta signup is worth about a third of a Google Search signup over 180 days. I built one model for Meta signups and one for Google Ads signups. 72 hours after a signup, the model predicts how much revenue that signup will bring over 180 days, in euros. I built the models on the signups from October 2025 to January 2026 and tested them on February and March 2026, which the models had not seen. On Meta, the third of test signups with the highest predicted value brought € 270 per signup over 180 days; the third with the lowest value brought € 7. Today every signup counts the same and brings € 103 on average. The value is sent to Meta through the Conversions API and to Google Ads through conversion upload; the data runs on Snowflake with Weld and dbt; the impact is proven with the platforms' own A/B tests on cost per paying customer.

Task 1 · Build a lead-value model

• Build a model that predicts the value of a signup early enough to send it to the ad platforms

What the model is. A formula that turns the facts we know about a signup 72 hours after signup into one number: the revenue that signup will bring over 180 days, in euros. In the dataset *revenue_180d_eur* equals plan price × months the customer stayed (1-6 months), so it rewards customers who stay, not only expensive plans.

Two models. One trained and used on Meta signups (2,555), one on Google Ads signups (2,292: *google_search* + *google_pmax*, two campaign types of the same account bidding on the same conversion). Organic and referral signups are not used at all: there is no platform to send their value to. I also tested one model on all 6,000 signups: it ranks signups just as well, but it predicts € 124 per Meta signup where the truth is € 103 (+20%), because the same behaviour pays less from Meta than from Google (calendar connected: 46% pay vs 67%). Separate models avoid this.

How it learns. The signups from October 2025 to January 2026 are the answer key: for each we know the facts at 72h *and* the outcome (*converted_paid_30d*, *revenue_180d_eur*). These two outcome columns are used only as the answers to learn from, never as inputs. Because 74% of signups bring € 0, the question is split in two steps. **Step 1, will they pay within 30 days?** A logistic regression: every fact gets a weight, the weights add up to a probability; the weights are found by comparing the signups that paid with those that did not. **Step 2, if they pay, how much?** A second formula with its own weights, trained only on the signups that paid, comparing those who brought more with those who brought less. **Value = step 1 × step 2.**

Three real Meta signups from the test months. A trades company of 6-20 with a business email that connected the phone and made 9 test calls: 93% × € 865 = **€ 802**. A one-person company that only connected the phone: 5% × € 409 = **€ 21**. A one-person automotive business on mobile that did nothing: 0.3% × € 136 = **€ 0**.

In production the formula is applied every day to the signups that turned 72h old. It is re-trained once a month on the latest signups whose outcomes are known, and replaced only if it beats the current formula on the last two months.

• Decide when you score a signup, and the trade-off between speed and accuracy

At 24h we know *phone_connected_24h* and *assistant_configured_24h*. At 72h we also know *test_calls_72h*, *calendar_connected_72h* and whether the signup already paid. Both moments fit the platforms' windows: Meta accepts events up to 7 days old, Google up to the conversion window we set (30 days). Revenue brought by the top 30% of signups: Meta 81% when scored at 24h, 85% at 72h; Google 65% at 24h, 74% at 72h. **Decision: 72 hours.** Waiting two more days adds 4 points on Meta and 9 on Google.

• Which features we used, which we left out, and why

Used, exactly as in the table: country, industry, company_size, device, business_email, signup_hour, signup_date (as weekday), phone_connected_24h, assistant_configured_24h, test_calls_72h, calendar_connected_72h. **Plus two features of our own:** *paid_by_72h* = True if *days_to_first_payment* ≤ 3, otherwise False; *plan_by_72h* = *plan_selected* for those signups, 'none' for everyone else. In the data every payer has a plan and every plan has a payment, so the two always match. 152 of the 1,559 payers had already paid by 72h.

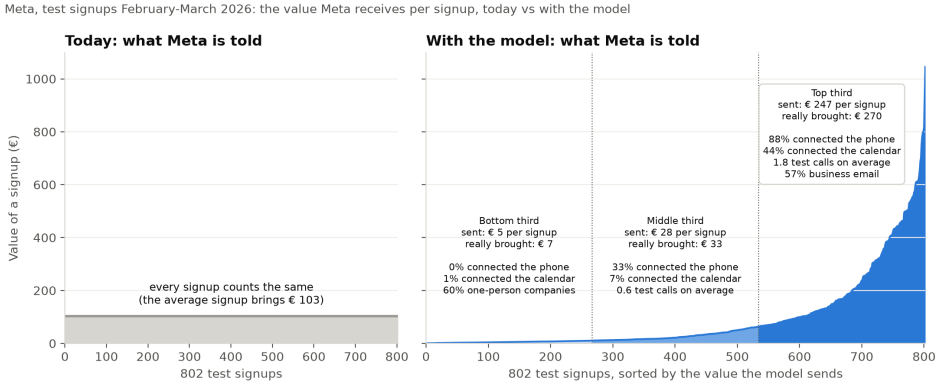
Left out: *minutes_used_7d*, *converted_paid_30d*, *revenue_180d_eur* (not known before 72 hours); *onboarding_call_booked* (no timestamp); *signup_id* (an ID); *plan_selected*, *days_to_first_payment*: these are filled in when the payment happens, and at 72 hours only 3% of signups have paid. In production the model would see 'none' and an empty date for 97% of new signups and would rate almost all of them as worthless. We use these two columns only to build *paid_by_72h* and *plan_by_72h*.

• How we validated it, and how much better it is than treating every signup the same

How we validated it. The file holds six months of signups with known outcomes. We built the models on the first four months (October 2025 to January 2026) and kept the last two (February and March 2026) aside. The split follows time, as in production: the model learns from the past and is applied to what comes next. Four months give enough examples to learn from (280 Meta payers), two months give enough to judge (802 Meta signups). The models then predicted a value for every February-March signup, and we compared it with the real *converted_paid_30d* and *revenue_180d_eur*. On average the Meta model predicted € 93 per signup and the real figure was € 103; Google Ads € 282 vs € 289. The euro number can be trusted.

How much better it is than treating every signup the same. Today Meta cannot tell good signups from bad ones and bids the same for all of them. With the model, the top third of signups is worth 2.6× the average signup and the bottom third is close to zero. Meta can bid accordingly: more for signups like the top third, less for signups like the bottom third, which means a lower cost per paying customer and a higher LTV per customer. Meta, February-March 2026, 802 test signups:

	Today: every signup counts the same	With the model: top third of signups	With the model: bottom third of signups
What Meta is told a signup is worth	the same for every signup	€ 247 on average	€ 5 on average
What they really brought, per signup	€ 103	€ 270	€ 7



Full tables, charts, the same comparison for Google Ads and the weights of all four formulas are in the notebook.

Task 2 · Feed it back to the ad platforms

- **Which event and value we send, and when**

One server-side event per signup, **QualifiedLead**, sent once when the signup turns 72h old, with *value* = predicted 180-day revenue and *currency* = EUR. The signup event stays as a secondary conversion for reporting. On day 30 we send the truth: Google gets a conversion adjustment with the real plan value, Meta gets a Purchase event, so the platforms learn from outcomes, not only forecasts.

- **Which APIs we use, and how we match users**

Meta: Conversions API. *event_name* QualifiedLead, *event_time* = scoring time (must be under 7 days old), *event_id* = signup_id so a retry never counts twice, *user_data* = email and phone hashed with SHA-256 (the platform compares hashes, never sees the raw email), *fbc* built from the fbclid in the ad click URL that we store at signup, *fbp* cookie, IP and user agent from the signup request. **Google Ads:** Google Ads API conversion upload keyed on the *gclid* (click ID from the ad URL, stored at signup), with *conversion_date_time*, value, EUR and *order_id* = signup_id against duplicates. If no click ID was stored (no consent, other device), fall back to enhanced conversions for leads with the hashed email. Matching needs consent to store click IDs and send hashed data; in DACH expect 60-70% of signups to match.

- **How this fits each platform's time windows, and the bidding strategy**

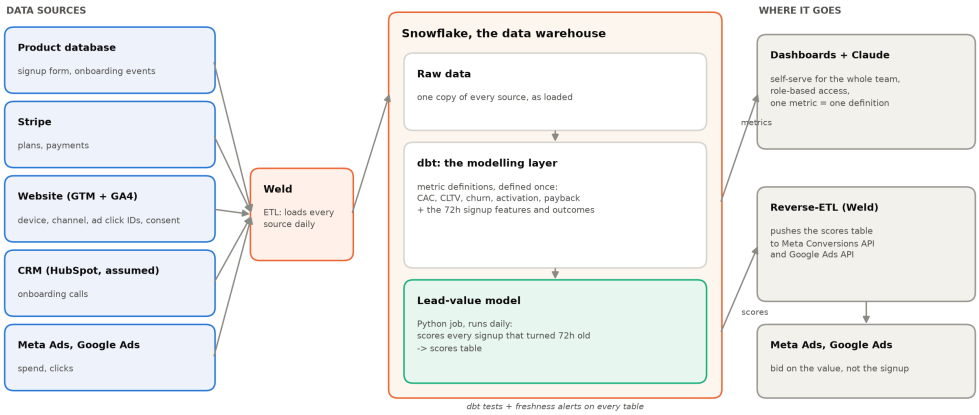
Google Ads: QualifiedLead becomes the primary conversion goal (signup secondary); conversion window 30 days, upload at 72h. Start with target CPA on QualifiedLead until a campaign has 30+ of them a month, then switch to Maximise conversion value with a target ROAS. Search and PMax use the same goal. **Meta:** run the campaign under the Sales objective with QualifiedLead as the conversion event; Meta unlocks value optimisation once 30+ events with a value arrive within 7 days (Meta sends ~100 signups a week, so ~40 qualified a week is realistic). If that threshold is not met, use Conversion Leads optimisation (CRM integration, lead stages) instead.

- **What happens to signups the model rates as low value**

They are sent too, with their low value (floor € 1). The platform learns from the spread between € 1 and € 800; dropping them hides the negative examples. In the target-CPA phase, 'qualified' means predicted value above the top-40% cut-off. Low-value signups get product onboarding, not more ad spend.

Task 3 · Data foundation

Which sources feed in: the product database (signup form, onboarding events), Stripe (plans, payments), the website through GTM and GA4 (device, channel, ad click IDs, consent), the CRM (onboarding calls; HubSpot assumed), and Meta Ads and Google Ads (spend, clicks). **Where the data lives:** Weld loads every source into Snowflake once a day. Inside Snowflake, dbt turns the raw tables into the numbers the business runs on: the metric definitions, defined once (CAC, CLTV, churn, activation, payback), plus the 72h signup features and outcomes the model needs. Every table has dbt tests and freshness alerts. **How the model runs:** a Python job runs once a day, scores every signup that has turned 72 hours old and writes the result to a scores table; the model is re-trained monthly. **How scores flow back out:** Weld's reverse-ETL pushes the scores table to the Meta Conversions API and the Google Ads API. The same warehouse serves the team's dashboards and ad-hoc questions through Claude, with role-based access. I have built this stack on BigQuery; the design is the same on Snowflake.



What I would build first, in the first 30 days. Days 1-7, data connectors. Connect Weld to the product database, Stripe, GA4, Meta Ads and Google Ads and land everything in Snowflake. Make sure the website saves the ad click IDs and the consent flag with every signup, otherwise nothing can be matched later. **Days 8-14, warehouse and ETL pipelines.** Daily loads, tests that every load is complete and on time, and one signups table that joins all sources. **Days 15-21, modelling layer in dbt.** Define the metrics once (CAC, CLTV, churn, activation, payback) and build the 72h feature table and the outcomes table. **Days 22-30, first signal to the platforms.** Send a simple rule-based value to Meta and Google Ads and check the match rates, then train the model on warehouse data and set up the A/B test from Task 4.

Task 4 · Prove the impact

Test setup: the platforms' own A/B tests, which split the audience cleanly: Meta Experiments with two identical campaigns, one optimising on Signup and one on QualifiedLead value; a Google Ads campaign experiment with a 50/50 budget split. Run 8 weeks: Meta yields ~70 paying customers a month, and the result must be visible 30 days after the last signup. **Decision metric: cost per paying customer at day 30** = ad spend ÷ customers who paid within 30 days. The scored arm wins if its cost is lower and the experiment report shows at least 90% confidence, which means: if the two setups really made no difference, chance alone would produce a gap this big less than one time in ten. Guardrail: the scored arm must still deliver at least 80% of the control arm's paying customers, so efficiency is not bought by shrinking volume. Predicted value per euro spent (visible at 72h) is only an early read; revenue per euro at day 90 confirms afterwards.

Other. The dataset is synthetic; in production revenue_180d is unknown for the last 6 months of signups, so training uses signups old enough to have outcomes. **Tooling:** Python 3.11 with pandas, scikit-learn (logistic and ridge regression, gradient boosting for comparison), matplotlib and Jupyter. Claude Code was used throughout as a pair programmer and sparring partner: profiling the data, first drafts of the code and charts, and challenging the choices (leakage, channel, separate vs pooled models, what the platforms' value field accepts). Decisions and the final text are mine; every number is reproduced by *src/run_experiments.py*.